DESIGN REPORT

LIBRARY RESOURCE MANAGEMENT SYSTEM

SESEIU24001 - Võ Châu Anh

---

1. Assumptions

These are the assumptions I used:

- Every resource has a unique ID that never changes after creation. I used `let id` for this.
- There are two member roles: Student and Lecturer. Lecturers get twice the loan period (`loanMultiplier = 2.0`) and a higher borrow limit (5 vs 3).
- Money is stored in VND as `Double` and formatted with a `vnd` helper.
- "Reference only" journals can be viewed but cannot be borrowed.
- Digital materials have limited concurrent seats (`maxUsers`) and a license expiry date. Because the license controls access automatically, I set `lateFeePerDay = 0` for digital materials — when the license expires, access is revoked, so a late fee would be meaningless.
- Fee model:
    - Refundable deposit (books and journals)
    - Non-refundable access fee (digital only)
    - Late fee per day (books and journals only)
- No backend — local sample data is enough for the demo. Sample data is loaded by `LibraryManager(sample: true)`; `LibraryManager.preview()` reuses the same initializer for SwiftUI previews.

2. Class identification

Class	Responsibility
`LibraryResource`	Base class. Holds shared data (`id`, `title`, `publicationYear`, `loans`) and defines the borrow/return API.
`PrintedBook`	Physical book. Adds author, ISBN, and short-loan flag.
`AcademicJournal`	Journal with volume/issue. Some are reference-only.
`DigitalMaterial`	Online resource with platform, license expiry, max concurrent users.

Member	Library user with id, name, role.
MemberRole (enum)	Student / Lecturer — controls borrow limit and loan multiplier.
LibraryManager	Owns all data. Provides add / borrow / return / search operations.

Supporting types (needed because the base classes alone are not enough):

- Loan (struct) — one active borrow with due date, deposit, and fee paid.
- ReturnReceipt (struct) — result of returning a resource.
- InfoItem (struct) — key-value row used by the UI.
- ResourceType (enum) — for filtering the list.
- LibraryError (enum) — all validation errors with clear messages.

3. Properties & Methods

Shared stored properties in LibraryResource:

- let id — immutable identity
- private(set) var title, private(set) var publicationYear — readable everywhere, writable only inside the class hierarchy
- private(set) var loans: [Loan] — the only state that changes on borrow/return

Computed properties in LibraryResource:

- isAvailable (loans.count < capacity)
- availableCopies (capacity - loans.count, clamped to ≥ 0)
- availabilityText — polymorphic string ("Available" / "Borrowed" / "2/3 seats free" / "In-library only" / "License expired")
- detailRows: [InfoItem] — polymorphic list of rows shown on the detail screen

Methods in LibraryResource:

- init(id:title:publicationYear:)
- loanDays(for member:) — base days × role multiplier, rounded, at least 1
- calculateLateFee(daysLate:) — max(0, days) × lateFeePerDay
- lateDays(for:on:) — day difference between due date and a given date
- isOverdue(_:on:) — convenience
- validateBorrow(by:on:) — checks isBorrowable, duplicate loan, availability
- borrow(by:on:) — final — the only way to add a loan
- giveBack(memberID:on:) — final — the only way to remove a loan; returns a ReturnReceipt

Overridable policy hooks (computed var with defaults in the base class):
category, typeName, icon, capacity, baseLoanDays, depositAmount, accessFee,

lateFeePerDay, isBorrowable, restrictionReason, actionVerb, returnVerb, searchableText, availabilityText, detailRows.

Subclass-specific stored properties (with access levels):

Class	Additional stored properties
PrintedBook	private(set) var author, private(set) var isbn, let isShortLoan
AcademicJournal	private(set) var volume, private(set) var issue, let isLendable
DigitalMaterial	private(set) var platform, private(set) var licenseExpiry, private(set) var maxUsers, plus computed isLicenseValid

private(set) means the value can be read from outside but only the class itself may change it. let means it is fixed at construction time. In practice, subclass metadata (author, ISBN, volume, issue, platform…) is created once and never edited — if the user needs to correct it, the resource is removed and re-added.

4. Inheritance & Polymorphism

Hierarchy:

```
LibraryResource
├── PrintedBook
├── AcademicJournal
└── DigitalMateria
```

Inherited (written once, used by all):

- Storage for id, title, publicationYear, loans
- borrow / giveBack — final, so subclasses cannot override or bypass them
- Late-fee formula (calculateLateFee)
- Loan-days calculation (loanDays(for:))
- Base rows of detailRows (type + year)

Complete override matrix (rows = overridable property, columns = subclass):

Property	PrintedBook	AcademicJournal	DigitalMaterial
category	.book	.journal	.digital

typeName	"Printed Book" / "(short loan)"	"Academic Journal"	"Digital Material"
icon	book.fill	doc.text.fill	laptopcomputer
capacity	—	—	maxUsers
baseLoanDays	28 / 7 (short)	7	3
depositAmount	0 / 50,000	100,000	—
accessFee	—	—	5,000
lateFeePerDay	2,000 / 5,000	5,000	0 (explicit)
isBorrowable	—	isLendable	isLicenseValid
restrictionReason	—	"in-library use only"	"the license has expired"
actionVerb	—	—	"Access"
returnVerb	—	—	"Release access"
searchableText	author + ISBN	vol/issue	platform
availabilityText	—	super + "In-library only"	super + "License expired"
detailRows	super + author, ISBN	super + vol, issue, lending	super + platform, expiry, users

A dash (—) means the subclass does not override that property and uses the base default. This is deliberate: `PrintedBook` inherits `accessFee = 0` because books have no access fee; `DigitalMaterial` inherits `depositAmount = 0` because digital items are not borrowed physically.

Polymorphism in practice: `LibraryManager.borrow(...)` works on a `LibraryResource` reference without ever checking the concrete type. Each subclass supplies its own `isBorrowable`, `capacity`, `baseLoanDays`, and `detailRows`. The same `borrow` call produces a

due date, deposit, and access fee appropriate to the actual type. The detail screen renders `r.detailRows` and the correct extra rows appear automatically.

When protocol/composition would be better: Swift does not allow multiple class inheritance. If a future resource is both digital and a journal (e.g., a digital journal with volume/issue and online access), a single class tree cannot express it. In that case, protocols like `Borrowable`, `OnlineAccessible`, and `HasIssue`, combined with composition, would be the cleaner design. I chose class inheritance here because the three current types share one unified loan lifecycle (`loans` array, `borrow`, `giveBack`) with the same state machine, and everything else is a matter of property values.

5. Encapsulation & Validation

Data protected from direct modification:

Data	Access modifier	Reason
`LibraryResource.id`, **`publicationYear`**	`let`	Identity / publication info never change
`LibraryResource.title`	`private(set) var`	Only editable via `LibraryManager.addResource` validation
`LibraryResource.loans`	`private(set) var`	State changes only through `borrow` / `giveBack`
`PrintedBook.author`, **`.isbn`**, **`.isShortLoan`**	`private(set)` / `let`	Book metadata is fixed at creation
`AcademicJournal.volume`, **`.issue`**, **`.isLendable`**	`private(set)` / `let`	Journal metadata is fixed at creation
`DigitalMaterial.platform`, **`.licenseExpiry`**, **`.maxUsers`**	`private(set)`	Controlled by vendor; updated only through manager operations
`Member.name`	`private(set)` + `setName` (trimmed, rejects empty)	Identity field

LibraryManager.resources / members	@Published private(set)	Manager owns the collection; views read only

Single gate for state change: borrow and giveBack are final, so subclasses cannot bypass them. External code must call LibraryManager.borrow(resourceID:...), which calls resource.borrow(...), which calls validateBorrow(...). There is no other way to append to or remove from loans.

Validation rules:

Rule	Where	Error case
ID / title not empty	addResource	emptyField
Year in 1450…current+1	addResource	invalidYear
maxUsers > 0	addResource	invalidCapacity
No duplicate ID	addResource	duplicateResource
Resource is borrowable	validateBorrow	notBorrowable
Member not already borrowing	validateBorrow	duplicateLoan
A slot is free	validateBorrow	notAvailable
Borrow limit not exceeded	LibraryManager.borrow	borrowLimitReached
Return date ≥ borrow date	giveBack	invalidReturnDate
Only the actual borrower can return	giveBack	notBorrowedByMember

6. SwiftUI implementation

Screen structure:

- ContentView — TabView with two tabs: Resources and My Loans. Injects LibraryManager(sample: true) as an @StateObject via .environmentObject.
- ResourcesView — search field + segmented filter by ResourceType. Uses MemberPicker (a Menu bound to manager.currentMemberID) to switch the current

user. List rows are `ResourceCard`, showing icon, title, ID chip, type name, availability pill and fee preview.
- `ResourceDetailView` — uses `resource.detailRows` (polymorphic) to render the type-specific block. Shows the policy / fee block, a Borrow button, and (when the current member holds a loan) a Return section with a DatePicker for a simulated return date plus a live estimate of late days and late fee. Every action result or rejection is shown in a `.messageAlert`.
- `LoansView` — lists the current member's active loans with due dates and "X days late" warnings.

Where polymorphic behavior is visible to the user:
- Card layout is identical, but the fee text differs — "Deposit 50,000 ₫" for a book, "Access fee 5,000 ₫" for digital.
- Button label changes — "Borrow" vs "Access"; return verb "Return" vs "Release access".
- Availability text — "Available / Borrowed" for books, "2/3 seats free" for digital, "In-library only" for reference-only journals, "License expired" for expired digital items.
- Detail screen extra rows come from the subclass's `detailRows` — book shows Author/ISBN, journal shows Volume/Issue, digital shows Platform/License expiry/Simultaneous users.

7. Conclusion

The design keeps shared behavior in `LibraryResource` (identity, loan lifecycle, late-fee formula) and pushes type-specific behavior into subclasses through overridable computed properties. State that must be protected (`id`, `title`, `loans`, all subclass metadata) is guarded by `let` or `private(set)`, and every change to the borrowed state goes through `borrow` / `giveBack` with full validation. The result is a small design that is easy to extend — a new resource type is just a new subclass with a few overrides — and easy to demonstrate in SwiftUI with clear valid/invalid feedback.